PRD: Hinglish Social Media Code-Mixed Machine Translation

Oct 10, 2026 · @Fenil

Summary

Train a Hinglish-to-English translation model from scratch, pre-trained on synthetic Hinglish and fine-tuned on real social media pairs, and show a measured improvement over a real-data-only baseline on a held-out PHINC test set.

This is a two-person NLP course project with a resume goal, due Oct 31, 2026. The output is a trained model, an ablation table, an error analysis and a small demo. The main model is a GRU with attention trained from scratch; a fine-tuned pretrained model is a stretch goal only.

Problem and background

Standard translation systems handle code-mixed Hinglish poorly, and the reference paper left the modelling work undone.

PHINC (Srivastava and Singh, IIT Gandhinagar) released 13,738 Hinglish social media sentences with human English translations. Its evaluation is thin:

- Baselines were scored on 50 sampled sentences only, using BLEU-1.
- Bing Translate scored 0.139, Google Translate 0.14, and the authors' chunking pipeline on top of Google Translate 0.153.
- No model was trained on the corpus.

The paper names the causes of failure: ambiguous language identification for Romanised words, non-standard spelling (jaldi, jldi, jldiii), informal sentence structure, missing punctuation and missing context. This project targets spelling variation and data scarcity directly.

Goals and non-goals

Goals

- Train a sequence-to-sequence translation model from scratch for Romanised Hinglish to English.
- Analyse the corpus first: vocabulary, sentence length, n-gram frequencies, out-of-vocabulary rate and Code-Mixing Index.
- Build a token-level language identifier, first rule-based and then n-gram based, and use it for the corpus analysis and the demo.
- Build a synthetic data pipeline that turns Hindi-English parallel text into noisy Romanised Hinglish.
- Evaluate on a full held-out test split with BLEU-4 and chrF, not 50 sentences and BLEU-1.
- Report an ablation that isolates the effect of synthetic pre-training, spelling noise and subword tokenisation.
- Ship a small demo where a user types Hinglish and gets English.

Non-goals

- Beating commercial translators or fine-tuned large models in absolute quality.
- English-to-Hinglish generation.
- Devanagari input, speech, or languages other than Hindi-English.
- Translating abusive content; PHINC annotators skipped these sentences, so there is no reference data.
- A production deployment.

Users and use cases

The users are teams that process Indian user-generated text with English-only tooling.

User	Use case
Customer support team	Read Hinglish tickets and chats in English before routing
Product analyst	Run English sentiment or topic models over Hinglish reviews
Content moderator	Review Hinglish posts without Hindi fluency
NLP researcher or student	A reproducible from-scratch baseline on PHINC

For the course, the evaluators are the second audience: they need a clear model, a clear metric and a working demo.

Course alignment

The project covers Units 2, 3 and 4 of the syllabus and the machine translation objective; speech processing in Unit 1 is out of scope.

Syllabus topic	Where it appears in the project
Corpus, corpus analysis, dataset preparation (Unit 2)	Corpus statistics on PHINC; building the synthetic corpus; train, validation and test splits
Handling raw text, pre-processing, normalisation (Unit 3)	Cleaning tweets, lowercasing, spelling normalisation and noise
Regular expressions (lab)	Masking mentions, hashtags and URLs
Tokenisation (Unit 1, lab)	Word-level versus BPE subword tokenisation, compared in runs A and B
Ambiguity handling (Unit 3)	Words such as is, me and to that are valid in both languages
Rule-based system for NLP (Unit 4)	Dictionary-based language tagger; rule-based romaniser
n-grams, TF-IDF, vectorisation, text classification (Unit 4, lab)	Character n-gram language identifier
Word embeddings (Unit 4)	Embeddings trained from scratch; nearest neighbours of spelling variants
Encoders and decoders, deep learning for NLP (Unit 4)	GRU encoder-decoder with attention
Applications: machine translation (objectives)	The translator and its demo
Case studies (Unit 4)	Reproducing and extending the PHINC paper

Stemming, POS tagging and NER are left out on purpose: standard tools for them assume monolingual, correctly spelled text and perform poorly on Romanised Hinglish. Named entities are covered only as a category in the error analysis.

Data

Real Hinglish-English pairs total about 22k across the two usable public sets, so the bulk of training data is synthetic.

Dataset	Size	Role
IIT Bombay English-Hindi corpus	1.49M segments, Devanagari	Source for synthetic Hinglish; sample 300k
PHINC	13,738 real pairs	Fine-tuning, validation and the test set
LinCE Eng-Hinglish (CALCS 2021)	8,060 train, 960 dev, 942 test	Extra real pairs for fine-tuning
Aksharantar / Dakshina	Large transliteration pair sets (size not checked)	Checking the romaniser's output

Overlap. The LinCE set is built from the CMU Hinglish DoG corpus, so the two are one source and are counted once. Its Hinglish side was written by translating English movie conversations, so it is less natural than PHINC's tweets and comes from a different domain.

Licences. PHINC is released under CC BY 4.0. The IIT Bombay corpus is free for non-commercial research, which covers a course project. LinCE's terms are shown at download and still need a check.

Splits. PHINC is split 80/10/10 into train, validation and test with a fixed seed. The test split is never used for training, tuning or vocabulary building. Near-duplicate sentences are removed across splits before training.

Synthetic generation.

1. Sample Hindi-English pairs and drop pairs outside 5-40 tokens, matching PHINC's length filter.
2. Romanise the Hindi side with a rule-based transliterator.

3. Apply spelling noise to the Romanised side: vowel drops (nahi to nhi), repeated characters (jaldi to jaldiii), common shortenings (hai to h), and lowercase.
4. Keep the English side unchanged as the target.

Known limitation. Step 2 produces Romanised Hindi, not true code-mixing. Real switching between Hindi and English inside a sentence is learned only from the real pairs.

Corpus analysis and language identification

Before any model is trained, the corpus is measured and every token is tagged as Hindi, English or other.

Corpus analysis. Reported for PHINC and for the synthetic corpus side by side, so the domain gap is visible in numbers:

- Sentence count, token count, vocabulary size and sentence-length distribution.
- Most frequent unigrams and bigrams.
- Out-of-vocabulary rate against an English dictionary.
- Spelling variation: groups of tokens that reduce to the same form after vowel removal, such as nahi, nhi and nahin.
- Code-Mixing Index over the whole corpus. The paper computed it by hand on 50 sentences and reported 77.

Language identification. Two versions, compared on labelled data:

1. Rule-based: a token is English if it is in an English dictionary, other if it is a mention, hashtag, URL or number, and Hindi otherwise. This is the paper's own method.
2. n-gram based: a logistic regression classifier on character n-gram TF-IDF features, trained on the LinCE Hindi-English language identification set. It handles the ambiguous words the dictionary gets wrong.

The tags are used in three places: the Code-Mixing Index, translation scores reported separately for low and high code-mixing sentences, and the demo, which colours each input word by language.

System design

The system runs in six steps, from data generation to a demo.

1. **Data pipeline.** Scripts that download, filter, romanise and add noise, producing versioned train files.

2. **Tokenisation.** One shared BPE vocabulary of about 8k subwords trained on the training text only. Subwords let the model relate spelling variants that a word-level vocabulary would treat as unrelated.
3. **Pre-training.** Train the model from random initialisation on the synthetic pairs.
4. **Fine-tuning.** Continue training on the real pairs at a lower learning rate, with early stopping on PHINC validation chrF.
5. **Inference.** Beam search with beam size 4. Mentions, hashtags and URLs are masked before translation and restored after, following PHINC's annotation rule that they stay unchanged.
6. **Demo.** A Streamlit page: text box in, translation out, with a few sample inputs.

Model specification

Component	GRU with attention
Architecture	Bidirectional GRU encoder, GRU decoder, attention
Layers	2 encoder, 2 decoder
Hidden size	256
Embeddings	Trained from scratch, shared source and target
Training	Adam, dropout 0.3, mixed precision

The GRU is the main model because the course teaches RNN, LSTM and GRU, so it is the easiest to defend in a viva. A Transformer comparison is a stretch goal.

These sizes are starting points chosen to train in a few hours on an RTX 3050 with 6 GB VRAM using mixed precision; tune them on validation data.

Stack. Python, PyTorch, SentencePiece for BPE, scikit-learn for the language identifier, sacreBLEU for scoring, Streamlit for the demo.

Evaluation

Success is a relative gain on the PHINC test split, not a high absolute score.

Metrics. BLEU-4 (sacreBLEU) and chrF on the full test split of about 1,370 sentences. chrF is the primary metric because it is more stable than BLEU on short, noisy sentences. BLEU-1 is also reported for comparison with the paper.

Ablation table to produce

Run	Model	Training data	Tokens	Spelling noise
A	GRU	Real only	Word-level	No
B	GRU	Real only	BPE	No
C	GRU	Synthetic then real	BPE	No
D (main)	GRU	Synthetic then real	BPE	Yes

Supporting results

- Language identifier: token accuracy and per-class F1 for the rule-based and n-gram versions on the LinCE test data.
- Run D scores split into low and high Code-Mixing Index sentences.
- Embedding check: nearest neighbours of ten common words in run D's embedding space, to see whether spelling variants such as jaldi and jldi sit together.

Stretch goals, in order. Attempt these only if runs A to D and the demo are done by 24 October.

1. A Transformer encoder-decoder trained on the run D recipe, to show the effect of architecture.
2. A small pretrained multilingual model such as mT5-small, fine-tuned with and without the synthetic data.

Success criteria

- Run D beats run B on both chrF and BLEU-4 on the test split.
- Run D beats run C, showing that spelling noise helps.
- An error analysis of 100 test sentences, tagged by the paper's failure categories (spelling variation, informal structure, missing context, language ambiguity).
- A human check of 50 outputs marked correct or needs change, the same method the paper used for its 0.84 translation quality figure.

Comparison with the paper. The paper's BLEU-1 figures come from a different 50-sentence sample, so they are context, not a like-for-like baseline. If time allows, score Google Translate on the same test split.

Deliverables and milestones

The plan fits three weeks for two people, ending on the 31 October 2026 submission.

Week	Person 1: corpus and language identification	Person 2: translation model
1, to 17 Oct	Downloads, cleaning, PHINC splits, corpus statistics, rule-based language tagger	Romaniser and noise scripts, 300k synthetic pairs, GRU training code, scoring script, runs A and B
2, to 24 Oct	n-gram language identifier and its evaluation, Code-Mixing Index, first Streamlit demo on the run B checkpoint	Pre-train on synthetic, fine-tune on real; runs C and D
3, to 31 Oct	Error analysis of 100 sentences, human check of 50, scores by code-mixing level, results section	Embedding check, demo switched to run D and polished, README, method section

If week 2 slips, drop the n-gram language identifier and keep the rule-based tagger. Runs A to D, the corpus analysis and the demo are the project.

Final deliverables

- GitHub repository with data scripts, training code, configs and fixed seeds.
- Trained model checkpoint and tokeniser.
- Results table and error analysis.
- Streamlit demo.
- Course report.

Risks and mitigations

Risk	Effect	Mitigation
Three-week timeline	Ablation left incomplete	The n-gram language identifier is dropped first; start GRU code in week 1 on real data

Risk	Effect	Mitigation
Demo left to the last days	No working application to show	First demo in week 2 on an early checkpoint
Domain gap: source corpus is formal text, PHINC is tweets	Pre-training helps less than hoped	Fine-tune on real pairs; report the gap as a finding
Low absolute scores	Demo output looks rough	Present relative gains and the error analysis; pick demo samples honestly
Noisy references: the paper's own check found 8 of 50 translations needing changes	Metric ceiling is lowered	Use chrF and the human check alongside BLEU
Synthetic text is not truly code-mixed	Model weak on mid-sentence switching	Rely on real pairs; no extra mixing step in this timeline
6 GB VRAM on the RTX 3050	Out-of-memory errors, slow runs	300k synthetic pairs, mixed precision, batch by token count, free Colab as a second machine
Test leakage through duplicates	Inflated scores	Deduplicate across splits before training
IIT Bombay corpus is non-commercial only	Cannot redistribute data or sell the model	Ship download scripts, not the data; say so in the README

Open questions

Decided

- Pretrained models are allowed, but the core project uses none; a pretrained run is a stretch goal. The course asks for an NLP application, so the demo is a required deliverable. The syllabus covers corpus analysis, n-grams, rule-based and machine learning methods, so those are now part of the project and the Transformer moves to a stretch goal.
- Submission is 31 October 2026 with a team of two.
- No architecture is required; the course teaches RNN, LSTM and GRU, so the GRU is the main model.

- Training runs on a laptop RTX 3050 with 6 GB VRAM, with free Colab as backup.
- Dataset sizes are now checked against the source papers; LinCE and CMU Hinglish DoG are one source.

Still open

- [ ] Confirm LinCE's licence terms at download.
- [ ] Decide who is Person 1 and who is Person 2.

Sources

- PHINC: A Parallel Hinglish Social Media Code-Mixed Corpus for Machine Translation
- CALCS 2021 Shared Task: Machine Translation for Code-Switched Data
- The IIT Bombay English-Hindi Parallel Corpus